

Prototipo Zoom

Sistema de Videoconferencia por Sockets

PC4 - Programación Orientada a Objetos

Universidad

Facultad de Ingeniería

Integrantes:

Docente:

24 de junio de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Tecnologías Utilizadas	2
4. Arquitectura del Sistema	3
5. Base de Datos	3
6. Diagrama de Casos de Uso	4
7. Diagrama de Secuencia	5
8. Descripción de Clases	6
8.1. Lado del Servidor	6
8.1.1. Protocolo	6
8.1.2. BaseDatos	6
8.1.3. AuthService	7
8.1.4. ManejadorCliente	7
8.1.5. Servidor	8
8.2. Lado del Cliente	8
8.2.1. MainApp	8
8.2.2. ClienteSocket	9
8.2.3. Usuario	9
8.2.4. Mensaje	10
8.2.5. LoginFrame	10
8.2.6. PantallaPrincipal	11
8.2.7. PantallaSala	11
8.2.8. PanelVideo	12
9. Protocolo de Comunicación	12
10. Manual de Instalación y Ejecución	12
10.1. Requisitos	12

10.2. Ejecución	12
10.3. Datos de Prueba	14
11. Conclusiones	14

1. Introducción

El presente documento describe el diseño e implementación de un **prototipo académico de videoconferencia** inspirado en Zoom, desarrollado en Python como parte del curso de Programación Orientada a Objetos (PC4).

El sistema emplea una arquitectura **cliente-servidor** basada en **sockets TCP** con un protocolo de mensajes en formato **JSON**. Cuenta con autenticación de usuarios, creación y gestión de salas virtuales, chat en tiempo real, transferencia de archivos y transmisión de video desde la cámara web.

2. Objetivos

- Aplicar los principios de la POO: herencia, encapsulamiento, polimorfismo.
- Implementar una arquitectura cliente-servidor mediante sockets TCP.
- Desarrollar un protocolo de comunicación propio basado en JSON.
- Integrar múltiples funcionalidades: login, salas, chat, archivos y video.
- Utilizar SQLite para la persistencia de información.
- Emplear concurrencia con **threading** para atender múltiples clientes.

3. Tecnologías Utilizadas

- **Lenguaje:** Python 3.10+
- **Interfaz gráfica:** Tkinter
- **Comunicación:** Sockets TCP
- **Protocolo:** JSON sobre TCP
- **Base de datos:** SQLite
- **Concurrencia:** threading
- **Video:** OpenCV o ffmpeg
- **Imágenes:** Pillow

4. Arquitectura del Sistema

El sistema sigue un modelo cliente-servidor. El servidor centraliza las conexiones y actúa como intermediario. Cada cliente se conecta mediante TCP y el servidor asigna un hilo independiente por conexión.

5. Base de Datos

La base de datos SQLite contiene las siguientes tablas:

Tabla	Columnas
Usuarios	IdUsuario, Nombres, Correo, PasswordHash, Rol, Activo
Salas	IdSala,CodigoSala, Nombre, IdHost, Estado, FechaCreacion
ParticipantesSala	IdParticipante, IdSala, IdUsuario, Estado, FechaIngreso
SolicitudesSala	IdSolicitud, IdSala, IdUsuario, Estado, FechaSolicitud
Mensajes	IdMensaje, IdSala, IdUsuario, Contenido, FechaEnvio
ArchivosCompartidos	IdArchivo, IdSala, IdUsuario, NombreArchivo, RutaArchivo, Tamano, FechaSubida

Cuadro 1: Tablas de la base de datos

6. Diagrama de Casos de Uso

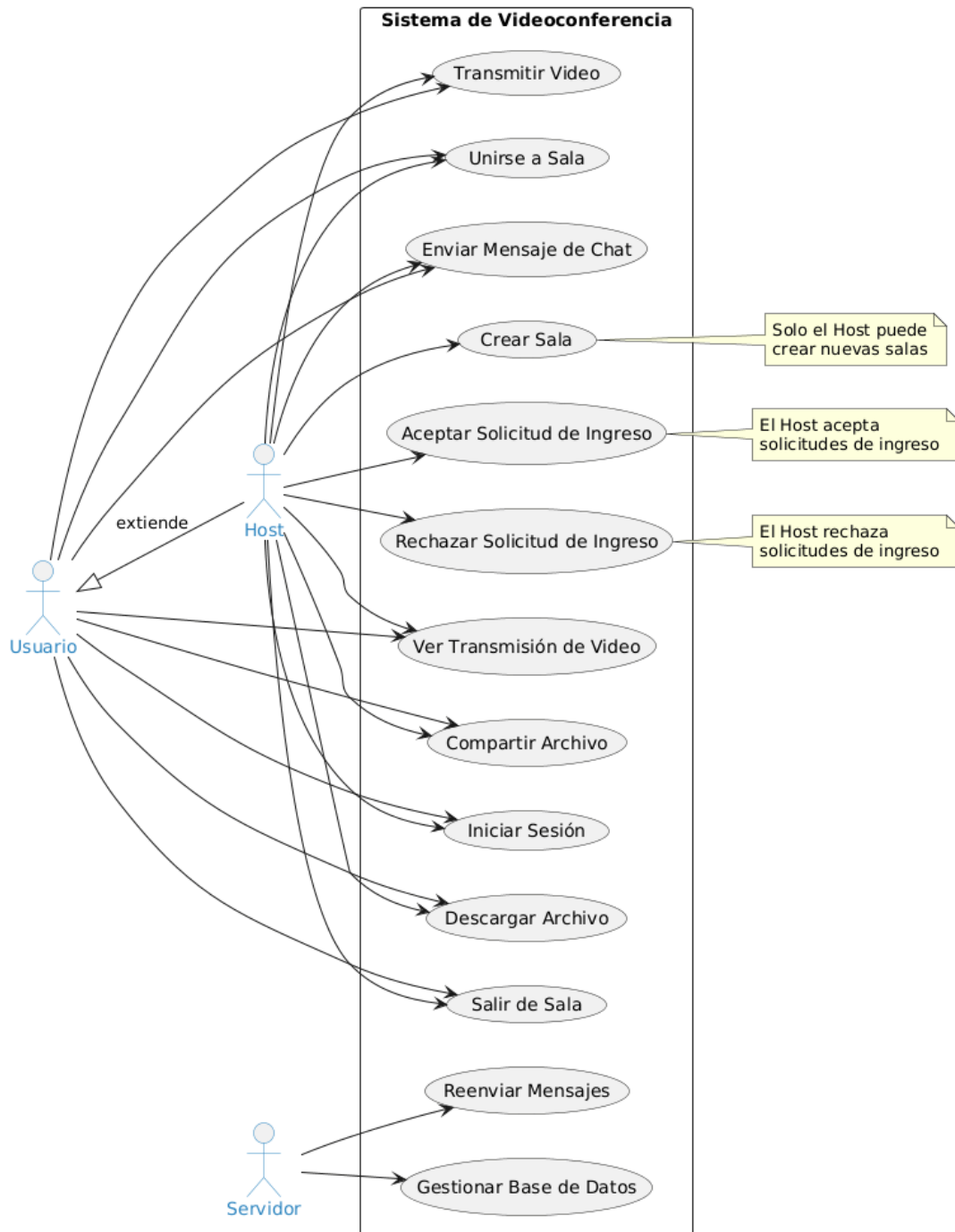


Figura 1: Diagrama de casos de uso del sistema

7. Diagrama de Secuencia

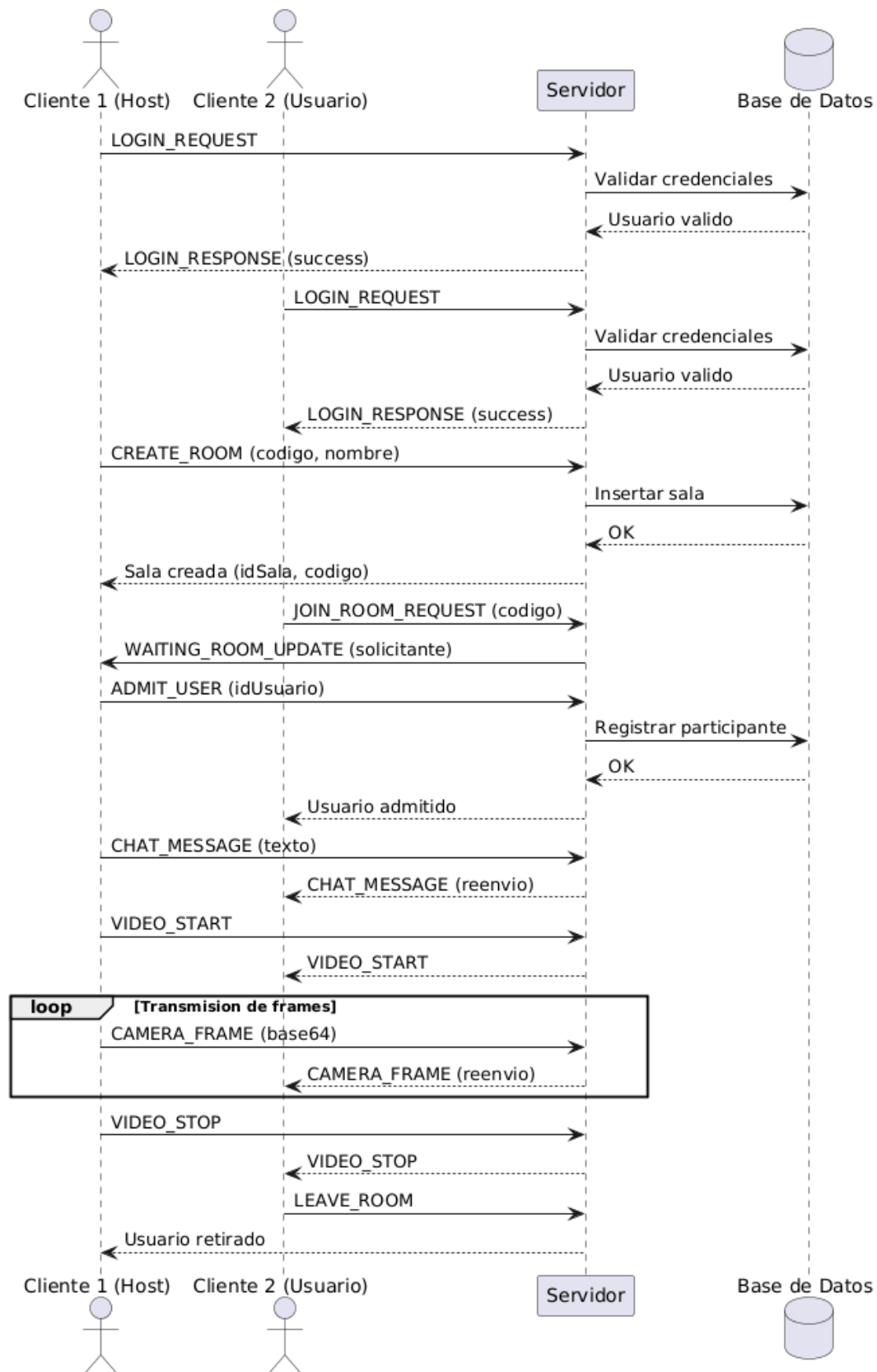


Figura 2: Diagrama de secuencia del flujo de la aplicación

8. Descripción de Clases

8.1. Lado del Servidor

8.1.1. Protocolo

Servidor/protocolo.py

Clase de constantes que define todos los tipos de mensaje del protocolo:

```
1 class Protocolo:
2     LOGIN_REQUEST = "LOGIN_REQUEST"
3     LOGIN_RESPONSE = "LOGIN_RESPONSE"
4     CREATE_ROOM = "CREATE_ROOM"
5     JOIN_ROOM_REQUEST = "JOIN_ROOM_REQUEST"
6     CHAT_MESSAGE = "CHAT_MESSAGE"
7     CAMERA_FRAME = "CAMERA_FRAME"
8     FILE_START = "FILE_START"
9     FILE_CHUNK = "FILE_CHUNK"
10    FILE_END = "FILE_END"
11    LEAVE_ROOM = "LEAVE_ROOM"
```

8.1.2. BaseDatos

Servidor/base_datos.py

Singleton que gestiona la conexión a SQLite. Inicializa el esquema y los datos de prueba.

```
1 class BaseDatos:
2     _instancia = None
3
4     def __new__(cls, *args, **kwargs):
5         if cls._instancia is None:
6             cls._instancia = super().__new__(cls)
7         return cls._instancia
8
9     def inicializar(self):
10        with open("crear_tablas.sql") as f:
11            self.conectar().cursor().executescript(f.read())
```


8.1.3. AuthService

Servidor/auth_service.py

Valida credenciales consultando la BD con hash SHA-256:

```
1 class AuthService:
2     def validar_login(self, correo, clave):
3         password_hash = BaseDatos.hash_clave(clave)
4         cursor.execute(
5             "SELECT IdUsuario, Nombres, Rol FROM Usuarios"
6             "WHERE Correo=? AND PasswordHash=? AND Activo=?1",
7             (correo, password_hash)
8         )
9         resultado = cursor.fetchone()
10        if resultado:
11            return {"status": "success", "idUsuario": ...}
12        return {"status": "error", "message": "Credenciales incorrectas"
13            }
```

8.1.4. ManejadorCliente

Servidor/manejador_cliente.py

Maneja la comunicación con un cliente en un hilo separado. El bucle principal lee mensajes JSON delimitados por \n y los despacha según su tipo:

```
1 class ManejadorCliente:
2     def iniciar(self):
3         buffer = ""
4         while self._conectado:
5             datos = self._conexion.recv(4096).decode("utf-8")
6             buffer += datos
7             while "\n" in buffer:
8                 linea, buffer = buffer.split("\n", 1)
9                 mensaje = json.loads(linea)
10                self._procesar_mensaje(mensaje)
11
12        def _procesar_mensaje(self, mensaje):
13            tipo = mensaje.get("type")
14            if tipo == Protocolo.LOGIN_REQUEST:
15                self._manejar_login(mensaje)
```

```
16         elif tipo == Protocolo.CREATE_ROOM:
17             self._manejar_crear_sala(mensaje)
18         elif tipo == Protocolo.CHAT_MESSAGE:
19             self._servidor.reenviar_a_sala(
20                 mensaje.get("roomCode"), mensaje, self)
```

8.1.5. Servidor

Servidor/servidor.py

Servidor TCP principal. Acepta conexiones y crea un hilo por cada cliente:

```
1 class Servidor:
2     def iniciar(self):
3         BaseDatos().inicializar()
4         self._socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5         self._socket.bind((self._host, self._puerto))
6         self._socket.listen()
7         while self._activo:
8             conexion, direccion = self._socket.accept()
9             manejador = ManejadorCliente(self, conexion, direccion)
10            hilo = threading.Thread(target=manejador.iniciar, daemon=
11                True)
12            hilo.start()
```

8.2. Lado del Cliente

8.2.1. MainApp

Cliente/main.py

Ventana principal Tkinter. Gestiona la navegación entre pantallas:

```
1 class MainApp(tk.Tk):
2     def _on_login_exitoso(self, usuario, cliente_socket):
3         self._usuario = usuario
4         self._cliente_socket = cliente_socket
5         self._mostrar_principal()
6
7     def _mostrar_sala(self, codigo_sala, es_host, id_sala):
8         self.geometry("900x600")
```

```
9         self._frame_actual = PantallaSala(  
10             self, self._usuario, self._cliente_socket,  
11             codigo_sala, es_host, id_sala, self._salir_sala  
12         )  
13         self._frame_actual.pack(fill=tk.BOTH, expand=True)
```

8.2.2. ClienteSocket

Cliente/cliente_socket.py

Abstracción del socket TCP con callbacks para despachar mensajes entrantes:

```
1 class ClienteSocket:  
2     def _escuchar(self):  
3         buffer = ""  
4         while self._conectado:  
5             datos = self._socket.recv(4096).decode("utf-8")  
6             buffer += datos  
7             while "\n" in buffer:  
8                 linea, buffer = buffer.split("\n", 1)  
9                 mensaje = json.loads(linea)  
10                tipo = mensaje.get("type")  
11                cb = self._callbacks.get(tipo)  
12                if cb:  
13                    cb(mensaje)  
14  
15     def enviar(self, datos):  
16         self._socket.send(  
17             (json.dumps(datos) + "\n").encode("utf-8"))
```

8.2.3. Usuario

Cliente/modelos/usuario.py

Modelo del usuario autenticado:

```
1 class Usuario:  
2     def __init__(self, id_usuario, nombres, correo, rol):  
3         self._id_usuario = id_usuario  
4         self._nombres = nombres  
5         self._correo = correo
```

```
6         self._rol = rol
7
8     def es_host(self):
9         return self._rol.lower() == "host"
```

8.2.4. Mensaje

Cliente/modelos/mensaje.py

Modelo de mensaje de chat con serialización a diccionario:

```
1 class Mensaje:
2     def a_dict(self):
3         return {
4             "type": "CHAT_MESSAGE",
5             "roomCode": self._sala_codigo,
6             "userId": self._id_usuario,
7             "userName": self._nombre_usuario,
8             "message": self._contenido
9         }
```

8.2.5. LoginFrame

Cliente/pantallas/login.py

Pantalla de inicio de sesión. Envía LOGIN_REQUEST al servidor:

```
1 class LoginFrame(tk.Frame):
2     def _procesar_login(self):
3         respuesta = self._cliente_socket.enviar_y_recibir({
4             "type": "LOGIN_REQUEST",
5             "correo": self._entrada_correo.get(),
6             "clave": self._entrada_clave.get()
7         })
8         if respuesta.get("status") == "success":
9             usuario = Usuario(respuesta["idUserio"],
10                                respuesta["nombres"], correo,
11                                respuesta["rol"])
12             self._on_login_exitoso(usuario, self._cliente_socket)
```

8.2.6. PantallaPrincipal

Cliente/pantallas/pantalla_principal.py

Menú principal con opciones de crear o unirse a una sala:

```
1 class PantallaPrincipal(tk.Frame):
2     def _crear_sala(self):
3         codigo = ''.join(random.choices(
4             string.ascii_uppercase + string.digits, k=6))
5         respuesta = self._cliente_socket.enviar_y_recibir({
6             "type": "CREATE_ROOM",
7             "nombre": f"Sala_{self._usuario.nombres}",
8             "codigo": codigo,
9             "idHost": self._usuario.id_usuario
10        })
11        if respuesta.get("status") == "success":
12            self._on_entrar_sala(codigo, True, respuesta["idSala"])
```

8.2.7. PantallaSala

Cliente/pantallas/pantalla_sala.py

Pantalla principal de la sala. Integra chat, video y transferencia de archivos:

```
1 class PantallaSala(tk.Frame):
2     def _enviar_mensaje(self):
3         texto = self._entrada_msg.get().strip()
4         self._cliente_socket.enviar({
5             "type": "CHAT_MESSAGE",
6             "roomCode": self._codigo_sala,
7             "userName": self._usuario.nombres,
8             "message": texto
9        })
10
11    def _enviar_archivo_thread(self, ruta, nombre, tamano):
12        with open(ruta, "rb") as f:
13            datos = base64.b64encode(f.read()).decode()
14        self._cliente_socket.enviar({
15            "type": "FILE_START", "roomCode": self._codigo_sala,
16            "fileName": nombre, "fileSize": tamano
17        })
```

```
18         for i in range(0, len(datos), 1500):
19             self._cliente_socket.enviar({
20                 "type": "FILE_CHUNK", "roomCode": self._codigo_sala,
21                 "fileName": nombre, "data": datos[i:i+1500]
22             })
```

8.2.8. PanelVideo

Cliente/pantallas/pantalla_video.py

Panel que muestra frames de video desde base64:

```
1 class PanelVideo(tk.Frame):
2     def actualizar_frame(self, datos_b64):
3         img_data = base64.b64decode(datos_b64)
4         img = Image.open(io.BytesIO(img_data))
5         img = img.resize((self._width, self._height), Image.LANCZOS)
6         self._imagen_tk = ImageTk.PhotoImage(img)
7         self._label_video.config(image=self._imagen_tk, text="")
```

9. Protocolo de Comunicación

Los mensajes se envían en JSON sobre TCP, delimitados por `\n`.

10. Manual de Instalación y Ejecución

10.1. Requisitos

- Python 3.10 o superior
- Pillow (instalar con `pip install -r requirements.txt`)
- Opcional: OpenCV (`pip install opencv-python`) o ffmpeg

10.2. Ejecución

1. Iniciar el servidor:

```
1 python -m Servidor.servidor
```

Tipo	Dirección	Campos
LOGIN_REQUEST	Cliente → Servidor	correo, clave
LOGIN_RESPONSE	Servidor → Cliente	status, idUsuario, nombres, rol
CREATE_ROOM	Cliente → Servidor	codigo, nombre, idHost
JOIN_ROOM_REQUEST	Cliente → Servidor	codigo, idUsuario
WAITING_ROOM_UPDATE	Servidor → Host	idSala, solicitanteId, solicitanteNombre
ADMIT_USER	Host → Servidor	idSala, idUsuario
REJECT_USER	Host → Servidor	idSala, idUsuario
CHAT_MESSAGE	Cliente → Sala	roomCode, userName, message
FILE_START	Cliente → Sala	roomCode, fileName, fileSize
FILE_CHUNK	Cliente → Sala	roomCode, fileName, data
FILE_END	Cliente → Sala	roomCode, fileName
CAMERA_FRAME	Cliente → Sala	roomCode, userName, data (base64)
VIDEO_START	Cliente → Sala	roomCode, userName
VIDEO_STOP	Cliente → Sala	roomCode, userName
LEAVE_ROOM	Cliente → Servidor	roomCode

Cuadro 2: Tipos de mensaje del protocolo

2. Iniciar uno o más clientes:

```
1 python -m Cliente.main
```

10.3. Datos de Prueba

Nombre	Correo	Clave	Rol
Luis Tasayco	luis@email.com	123456	Host
Ana Torres	ana@email.com	123456	Usuario
Carlos Ruiz	carlos@email.com	123456	Usuario

Cuadro 3: Usuarios de prueba

11. Conclusiones

- Se implementó un prototipo funcional de videoconferencia con arquitectura cliente-servidor.
- Los sockets TCP con protocolo JSON proporcionan una base sólida para comunicación en tiempo real.
- La POO permitió organizar el código en clases especializadas con responsabilidades claras.
- El patrón Singleton garantiza una única instancia de conexión a la base de datos.
- La concurrencia con hilos permite atender múltiples clientes simultáneamente.
- La transmisión de video se logró mediante captura, compresión JPEG y codificación base64.